

02456 Deep Learning

Exercises Week 3

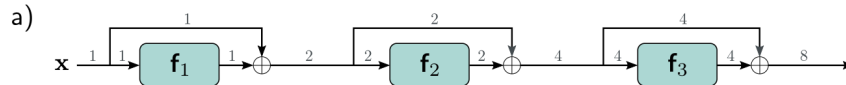
Viggo Starcke
s250690@dtu.dk

Prince 9.4 Derive the loss function for multiclass classification when we use label smoothing so that the target probability distribution has 0.9 at the correct class and the remaining probability mass of 0.1 is divided between the remaining $D_o - 1$ classes;

If we have a target vector for e.g. 3 classes it would look like $[0.9, 0.05, 0.05]$ if target class 1. If we generalize this further; for c being the correct class we have the term $y_c \log(\hat{y}_c)$ for the correct class and $y_k \log(\hat{y}_k)$ for the rest. Thus the general cross entropy loss function for multiclass classification, $L = -\sum_{k=1}^{D_o} y_k \log(\hat{y}_k)$, becomes

$$L = -\left(y_c \log(\hat{y}_c) + \sum_{k \neq c} y_k \log(\hat{y}_k)\right)$$

Prince 11.4 Explain why the values in the two branches of the residual blocks in figure 11.6a are uncorrelated. Show that the variance of the sum of uncorrelated variables is the sum of their individual variances;



The two branches of the residual blocks are the identity branch and residual branch. The identity branch is the one skipping over the function boxes, and is just the identity function on its input. The residual branch is the one labelled with boxes, f_1, f_2, f_3 . Their output is some transformation of the input, $f_n(x)$. Since the weights in the residual branch is sampled randomly at initialization from some distribution centered around 0, and thus independent of the input x , the resulting output $f_n(x)$ is not in a linear relationship with the original input x . This means the identity of the input is uncorrelated to this residual output.

The formula for variance of any two random variables is $\text{Var}(A + B) = \text{Var}(A) + \text{Var}(B) + 2\text{Covar}(A, B)$. But since the covariance between uncorrelated variables is 0. The final formula becomes $\text{Var}(A + B) = \text{Var}(A) + \text{Var}(B)$.

Exam 2025 Q4 Consider constructing a fully connected network with two hidden layers of size 128 that takes a 10×10 image with three channels as input and classifies it into five classes. All internal nodes use the ReLU activation function. The input tensor has shape $[B, 3, 10, 10]$, where B is the batch size, and `nn` has been imported from `torch`. Which one of the following PyTorch code snippets implements such a network correctly?

A. `nn.Sequential(
 nn.Linear(3, 10, 10, 128),`

- ```

 nn.ReLU(),
 nn.Linear(128, 128),
 nn.ReLU(),
 nn.Linear(128, 5),
 nn.Softmax(dim=1)
)

B. nn.Sequential(
 nn.Flatten(128),
 nn.ReLU(),
 nn.Linear(128, 128),
 nn.ReLU(),
 nn.Linear(5),
 nn.Softmax(dim=1)
)

C. nn.Sequential(
 nn.Linear(300, 128),
 nn.ReLU(),
 nn.Linear(128, 128),
 nn.ReLU(),
 nn.Linear(128, 5),
 nn.Softmax(dim=1)
)

D. nn.Sequential(
 nn.Flatten(300, 128),
 nn.ReLU(),
 nn.Linear(128, 128),
 nn.ReLU(),
 nn.Linear(128, 5),
 nn.Softmax(dim=1)
)

```

C is the correct. The input feature is a picture of  $10 \times 10$  pixels with each pixel having 3 different colour options (one imagines those being blue, red, and yellow). Thus the input vector has 300 data points. This disqualifies immediately option A and B. The difference between the remaining options C and D is the first layer function. Option D uses `nn.Flatten`, which just reshapes the vector dimensions without altering the underlying data or applying any weights. This is not what we want from our initial layer. So option C is correct one.

**Exam 2025 Q7** Consider a fully connected neural network with one hidden layer of size 32 that maps an input tensor of size `[B, 10]` to 3 classes. The loss is computed as `loss(model(input), target)`, where `target` is a tensor of class indices of size `[B]`. Which one of the following code snippets correctly defines the network and the loss?

- ```

A.    model = nn.Sequential(
        nn.Linear(10, 32),
        nn.ReLU(),
        nn.Linear(32, 3)
    )
    loss = nn.CrossEntropyLoss()

```

- B.

```
model = nn.Sequential(
    nn.Linear(10, 32),
    nn.ReLU(),
    nn.Linear(32, 3),
    nn.Softmax(dim=1)
)
loss = nn.CrossEntropyLoss()
```
- C.

```
model = nn.Sequential(
    nn.Linear(10, 32),
    nn.ReLU(),
    nn.Linear(32, 3)
)
loss = nn.Softmax(dim=1)
```
- D.

```
model = nn.Sequential(
    nn.Linear(10, 32),
    nn.ReLU(),
    nn.Linear(32, 1)
)
loss = nn.CrossEntropyLoss()
```

A is the correct. Option C is incorrect as it assigns an activation function (`nn.Softmax`) as the loss function. Option D is incorrect as it only outputs to 1 class instead of 3. Option A and B are similar on all points except B applies the `nn.Softmax` layer to the NNs output before passing it to the loss function. Since `nn.CrossEntropyLoss()` handles normalisation through a log-softmax and a negative log-likelihood loss prior softmax application can actually corrupt the learning process, we say option A is therefore only correct option.

Exam 2025 Q9 Consider a PyTorch implementation of a fully connected neural network with a single hidden layer of size 16 that takes an input tensor of shape `[B, 5]`. The network's output is used to parametrise the mean and standard deviation (or a parameter from which it can be computed) of a Gaussian distribution. The loss function computes the negative log probability of the target tensor `target` of shape `[B]` up to a normalisation constant. Recall that the Gaussian negative log-likelihood can be written as

$$-\log p(t|\mu, \sigma) = \log \sigma + \frac{1}{2} \left(\frac{t - \mu}{\sigma} \right)^2$$

Which one of the following code snippets implements this correctly?

- A.

```
model = nn.Sequential(nn.Linear(5,16), nn.ReLU(), nn.Linear(16,1))
def loss(y, target):
    return ((y-target)**2).mean()
```
- B.

```
model = nn.Sequential(nn.Linear(5,16), nn.ReLU(), nn.Linear(16,2), nn.Softplus())
def loss(y, target):
    return (torch.log(y[:,1])+0.5*((target-y[:,0])/y[:,1])**2).mean()
```
- C.

```
model = nn.Sequential(nn.Linear(5,16), nn.ReLU(), nn.Linear(16,2))
def loss(y, target):
    return (torch.log(y[:,1])+0.5*((target-y[:,0])/y[:,1])**2).mean()
```

```

D. model = nn.Sequential(nn.Linear(5,16), nn.ReLU(), nn.Linear(16,2))
def loss(y, target):
    return (y[:,1]+0.5*((target-y[:,0])/torch.exp(y[:,1]))**2).mean()

```

D is the correct. A Gaussian distribution uses two features; the mean μ and the standard deviation σ . A is incorrect because it only outputs to 1 output feature. The standard deviation can not be a negative number, $[0, \infty)$. Option C does not account for this as `nn.Linear(16,2)` can easily output a negative number for this, and no activation function is applied to cover this up. Option B uses `nn.Softplus` for this, and option D alters the mathematical operations such that the logarithmic operation in the first term is substituted with the exponential of the denominator in the second term. Using `nn.Softplus` forces all output features into positive numbers which is not desired for the mean, which should range from $(-\infty, \infty)$. So option B is incorrect.